

Nesne Yönelimli Programlama

Hafta 3: Fonksiyonlar, Bellek Mimarisi ve Modülerlik

Dr. Hüseyin Yüce

Marmara Üniversitesi Teknoloji Fakültesi

Mekatronik Mühendisliği Bölümü

Hafta 3: Teorik ve Kavramsal Hedefler

Bu hafta hedeflenen temel kavramsal kazanımlar:

- **Fonksiyon Tanımlama ve Bildirim Mimarisi:**
 - Fonksiyon prototipleri, imza (*signature*) kavramı ve arayüz-uygulama ayrımı
 - Derleyicinin tek geçişli (*single-pass*) yapısı ve bildirim gereksinimi
- **Parametre Aktarım Yöntemleri ve Bellek Yönetimi:**
 - Değerle aktarım (*Pass by Value*), işaretçi ile aktarım (*Pass by Pointer*)
 - Referans ile aktarım (*Pass by Reference*) ve `const` referans güvencesi (`const T&`)
 - Varsayılan argümanlar (*Default Arguments*) kuralları
- **Fonksiyon Aşırı Yükleme ve Çözümleme:**
 - Fonksiyon aşırı yükleme (*Function Overloading*) ve İsim Bozma (*Name Mangling*)
 - Belirsizlik (*Ambiguity*) durumları ve tip dönüşüm mekanizmaları

Hafta 3: Uygulamalı ve Mekatronik Hedefler

Laboratuvar oturumu ve sistem mimarisi kazanımları:

- Bellek Anatomisi ve Yığın Çerçevesi:
 - Çağrı Yığını (*Call Stack*) ve Aktivasyon Kaydı (*Stack Frame*)
 - Yerel değişkenlerin ömrü (*lifetime*), kapsamı (*scope*) ve yığın taşması (*stack overflow*)
- Performans ve Kod Büyümesi:
 - `inline` fonksiyonlar ve fonksiyon çağrı yükü (*overhead*)
 - Makrolar (`#define`) ile `inline` fonksiyonların güvenlik ve tip farkları
- Modüler Kod Mimarisi ve Çoklu Dosya Yönetimi:
 - Başlık dosyaları (`.h`) ve kaynak dosyaları (`.cpp`) ayrımı
 - Başlık muhafızları (`#ifndef` , `#pragma once`) ve dairesel bağımlılıklar
- Mekatronik Vaka Analizi:
 - Endüstriyel robot eklem kinematiği ve telemetri filtre kütüphanesi tasarımı

Motivasyon: Neden Fonksiyonlar ve Modülerlik?

Karmaşık mekatronik yazılımları binlerce satırlık monolitik bloklar halinde yazılamaz:

- Spagetti Kod Problemi: Tüm mantığın `main()` içinde toplandığı kodlar test edilemez, hata ayıklanamaz ve bakım yapılamaz hale gelir.
- DRY Prensipleri (Don't Repeat Yourself): Bir motor kontrol algoritmasını veya kinematik dönüşümü her eksen için tekrar yazmak yerine tek bir fonksiyona bağlarız.
- Soyutlama (Abstraction): Bir fonksiyonu çağıran mühendis, fonksiyonun iç matematiğini bilmek zorunda kalmadan arayüzü (*interface*) kullanabilmelidir.
- Ekip Çalışması ve Modülerlik: Büyük projelerde yazılımcılar farklı `.cpp` modüllerini bağımsız geliştirip bağlayıcı (*linker*) ile birleştirirler.

Mühendislik İlkesi: İyi bir fonksiyon tek bir işi yapmalı (Single Responsibility), onu kusursuz yapmalı ve açık bir arayüze sahip olmalıdır.

Fonksiyonun Temel Anatomisi

C++ dilinde bir fonksiyon; geri dönüş tipi, adı, parametre listesi ve gövdesinden oluşur:

```
1 // [Dönüş Tipi] [Fonksiyon Adı]([Parametre Listesi])
2 double torkHesapla(double kuvvet, double yaricap) {
3     // Fonksiyon Gövdesi
4     double tork = kuvvet * yaricap;
5     return tork; // Geri dönüş ifadesi
6 }
```

Bileşenler:

- Geri Dönüş Tipi: Fonksiyonun hesaplayıp çağıran koda ilettiği değer in veri tipi ('void', 'int', 'double' vb.).
- İmza (Signature): Fonksiyon adı ve parametrelerinin tip sıralaması.
- Gövde (Body): Süslü parantezler { ... } arasındaki algoritmik blok.

Geri Dönüş Tipleri: `void` ve Değer Döndürenler

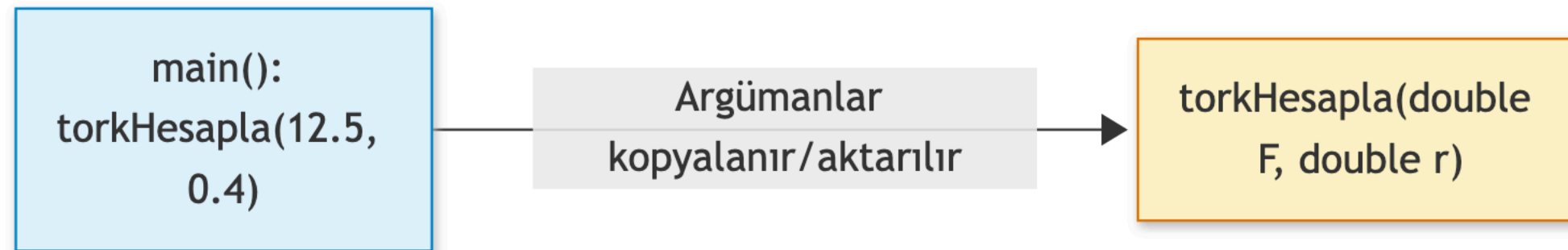
Fonksiyonlar işlevlerine göre değer üretebilir veya yalnızca eylem gerçekleştirebilir:

- Değer Döndüren Fonksiyonlar (`non-void`):
 - `return` ifadesi ile hesaplanan sonucu çağırان koda geri iletir.
 - Dönüş tipiyle `return` edilen değerin tipi derleme zamanında uyuşmalıdır.
- Değer Döndürmeyen Fonksiyonlar (`void`):
 - Sonuç üretmez; ekrana çıktı basma, motor pinini tetikleme gibi yan etkiler (*side effects*) gerçekleştirir.
 - İsteğe bağlı olarak fonksiyonu erken sonlandırmak için parametresiz `return;` kullanılabilir.

```
1 void motoruDurdur(int motorPin) {  
2     if (motorPin < 0) return; // Erken çıkış (Guard Clause)  
3     // Donanım pinini LOW seviyesine çek  
4 }
```

Terminoloji: Parametre ve Argüman Ayrımı

Mühendislik dokümantasyonunda bu iki terim sıklıkla karıştırılır ancak farklıdır:



- **Parametre (Formal Parameter):**

- Fonksiyon bildiriminde ve tanımında yer alan, gelecek veriyi karşılayan değişken kalıplarıdır.
- Örnek: `double torkHesapla(double kuvvet, double yaricap)` ifadesindeki `kuvvet` ve `yaricap`.

- **Argüman (Actual Argument):**

- Fonksiyon çağrılırken fonksiyona aktarılan somut değer veya değişkenlerdir.
- Örnek: `torkHesapla(150.0, 0.25)` çağrısındaki `150.0` ve `0.25`.

Fonksiyon Prototipleri (Forward Declaration)

C++ derleyicisi kaynak kodları yukarıdan aşağıya tek geçişte (*single pass*) okur:

- Eğer bir fonksiyon tanımlanmadan önce çağrılırsa derleyici hata verir (`identifier not found`).
- Fonksiyonun tüm gövdesini `main` 'den önce yazmak kod hiyerarşisini karmaşıklaştırır.

```
1 // Fonksiyon Prototipi (Bildirim - Deklarasyon)
2 double gerilimHesapla(double akim, double direnc);
3
4 int main() {
5     // Derleyici prototipi bildiği için çağrıya izin verir
6     double v = gerilimHesapla(2.5, 10.0);
7     return 0;
8 }
9
10 // Fonksiyon Tanımı (Gövde - Implementasyon)
11 double gerilimHesapla(double akim, double direnc) {
12     return akim * direnc;
13 }
```

Prototip Mimarisi: Bildirim ve Tanım Ayrımı

Fonksiyon prototipleri modüler yazılım tasarımının temel omurgasıdır:

- Prototipte Parametre İsimleri:
 - Derleyici için yalnızca tipler şarttır: `double gerilimHesapla(double, double);`
 - Ancak dokümantasyon ve okunabilirlik için parametre isimleri daima yazılmalıdır.
- Sözleşme (Contract) Modeli:
 - Prototip bir "sözleşme"dir: "Bu isimde bir fonksiyon var, bu parametreleri alır ve bu tipi döndürür."
 - Çağırılan kod gövdenin nasıl çalıştığıyla ilgilenmez; yalnızca sözleşmeye bakar.
- Bağlayıcı (Linker) Görevi:
 - Derleyici çağrı noktasında yer tutucu adres bırakır.
 - Fonksiyonun gerçek adresi bağlayıcı aşamasında çözülür.

C++'ta 3 Temel Parametre Aktarım Yöntemi

C++ dilinde argümanlar fonksiyona üç farklı felsefeyle aktarılabilir:

Yöntem	Sözdizimi	Bellek Davranışı	Orijinal Veri Değişir mi?
Değerle Aktarım (<i>Pass by Value</i>)	<code>void f(int x)</code>	Değerin kopyası stack'e çıkarılır	HAYIR
İşaretçiyle Aktarım (<i>Pass by Pointer</i>)	<code>void f(int* ptr)</code>	Değişkenin adresi fonksiyona verilir	EVET (deferans ile)
Referansla Aktarım (<i>Pass by Reference</i>)	<code>void f(int& ref)</code>	Değişkenin takma adı (<i>alias</i>) kullanılır	EVET (doğrudan)

C vs Modern C++: C dilinde yalnızca değerle ve işaretçiyle aktarım vardır. Referans mekanizması C++'ın getirdiği en güçlü ve güvenli yeniliklerden biridir.

1. Değerle Aktarım (Pass by Value)

Argümanın değeri kopyalanarak fonksiyonun yerel parametresine atanır:

```
1 void sifirla(int a) {  
2     a = 0; // Yalnızca 'sifirla' fonksiyonunun yerel kopyası değişir  
3 }  
4  
5 int main() {  
6     int sayi = 100;  
7     sifirla(sayi);  
8     // sayi DEĞİŞMEZ, hala 100'dür!  
9     std::cout << "Sayi: " << sayi << std::endl; // Çıktı: 100  
10    return 0;  
11 }
```

- **Güvenlik Avantajı:** Çağrılan fonksiyon çağıranın verisini kazara bozamaz.
- **Performans Dezavantajı:** Büyük veri yapıları (`struct` , 1000 elemanlı dizi, nesneler) kopyalanırken bellek ve işlemci israfına yol açar.

2. İşaretçi ile Aktarım (Pass by Pointer)

Fonksiyona değişkenin değeri yerine bellekteki adresi (`&sayi`) iletilir:

```
1 void sifirlaPtr(int* ptr) {  
2     if (ptr != nullptr) {  
3         *ptr = 0; // Adresteki hücreye gidilerek orijinal veri sıfırlanır  
4     }  
5 }  
6  
7 int main() {  
8     int sayi = 100;  
9     sifirlaPtr(&sayi); // sayi'nin adresi aktarılır  
10    // sayi DEĞİŞİR, artık 0'dır!  
11    std::cout << "Sayi: " << sayi << std::endl; // Çıktı: 0  
12    return 0;  
13 }
```

- Adres ile Erişim: Kopyalama maliyeti yoktur (yalnızca 8 bayt adres taşınır).
- Güvenlik Riski: İşaretçi `nullptr` veya geçersiz bir adres gösterebilir; her zaman null denetimi gerekir.

3. Referans ile Aktarım (Pass by Reference)

Referans (`&`), orijinal değişkenin bellekteki doğrudan takma adıdır (alias):

```
1 void sifirlaRef(int& ref) {  
2     ref = 0; // Orijinal 'sayi' değişkenine doğrudan erişilir  
3 }  
4  
5 int main() {  
6     int sayi = 100;  
7     sifirlaRef(sayi); // DİKKAT: &sayi değil, doğrudan 'sayi' yazılır!  
8     // sayi DEĞİŞİR, artık 0'dır!  
9     std::cout << "Sayi: " << sayi << std::endl; // Çıktı: 0  
10    return 0;  
11 }
```

- Sözdizimi Temizliği: Pointer gibi `*` veya `&` referans operatörlerine gerek kalmaz.
- Null Güvenliği: Referanslar boş (`null`) olamaz, tanımlandığı anda bir değişkene bağlanmak zorundadır.

Değer Değiştirme (Swap) Algoritması Karşılaştırması

İki değişkenin değerini takas eden fonksiyon üzerinden iki yaklaşım:

```
1 // C Tarzı (Pointer ile Swap)
2 void takasPtr(int* a, int* b) {
3     int gecici = *a;
4     *a = *b;
5     *b = gecici;
6 }
7
8 // C++ Tarzı (Referans ile Swap - Tercih Edilen)
9 void takasRef(int& a, int& b) {
10    int gecici = a;
11    a = b;
12    b = gecici;
13 }
```

Çağrı Sözdizimi Farkı:

Pointer çağrısı: `takasPtr(&x, &y);` (adres operatörü zorunlu)

Referans çağrısı: `takasRef(x, y);` (doğal değişken kullanımı)

const Referans ile Aktarım (const T&)

Modern C++ programlamada en yaygın ve en tavsiye edilen aktarım standardıdır:

```
1 struct TelemetriPaketi {
2     double sicaklik;
3     double basinc;
4     double ivme[3];
5     char mesaj[256];
6 };
7
8 // Sıfır kopyalama maliyeti + Salt okunur veri güvenliği
9 void telemetriYazdir(const TelemetriPaketi& paket) {
10     // paket.sicaklik = 0; // DERLEME HATASI! const koruması devrede.
11     std::cout << "Sicaklik: " << paket.sicaklik << std::endl;
12 }
```

- Bellek Kazancı: Yüzlerce baytlık yapı kopyalanmaz, yalnızca 64-bit adres gizlice taşınır.
- Salt Okunur Güvence: Fonksiyon gövdesinde verinin yanlışlıkla değiştirilmesi derleme zamanında engellenir.

• Geçici Nesne Kabulü: `const&` , sağ-değerlere (*r-value*, sabitlere) de bağlanabilir (`f(10)` geçerlidir).

Mühendislik Karar Matrisi: Hangi Yöntem Ne Zaman?

Parametre tipi seçerken aşağıdaki rehberi uygulayınız:

- Değerle Aktarım (`T`):
 - `int` , `float` , `char` , `bool` gibi temel ve küçük türler (≤ 8 bayt).
 - Fonksiyon içinde kopyanın bağımsız değiştirilmesi gerekiyorsa.
- Referansla Aktarım (`T&`):
 - Fonksiyonun çağıran koddaki değişkeni doğrudan güncellemesi gerekiyorsa (Çıktı parametresi).
- `const` Referansla Aktarım (`const T&`):
 - Sınıflar, yapılar (`struct`), `std::string` , `std::vector` gibi büyük türler.
 - Veri sadece okunacaksa modern C++'ın bir numaralı standardıdır.
- İşaretçiyle Aktarım (`T*` veya `const T*`):
 - Parametrenin "yok" (`nullptr`) olabilme ihtimali varsa (opsiyonel parametre).

Dizilerin Fonksiyonlara Aktarımı: Dizi Bozunması

C++'ta geleneksel diziler fonksiyonlara aktarıldığında boyutu kaybolur:

- Dizi adı, ilk elemanın adresine bozunur (*Array-to-Pointer Decay*).
- Bu nedenle dizinin boyutu mutlaka ayrı bir parametre olarak verilmelidir.

```
1 // int arr[] yazımı, derleyici için aslında int* arr demektir!
2 double ortalamaHesapla(const double arr[], int boyut) {
3     double toplam = 0.0;
4     for (int i = 0; i < boyut; ++i) {
5         toplam += arr[i];
6     }
7     return (boyut > 0) ? (toplam / boyut) : 0.0;
8 }
9
10 int main() {
11     double sensorVerileri[5] = {12.2, 14.5, 11.8, 15.0, 13.1};
12     double ort = ortalamaHesapla(sensorVerileri, 5);
13     return 0;
14 }
```

Varsayılan Argümanlar (Default Arguments)

Fonksiyon çağrılırken bir argüman girilmezse varsayılan bir değer alması sağlanabilir:

```
1 // Prototipte veya bildirimde tanımlanır
2 void motorHizAyarla(int motorId, int pwm = 128, bool frenAktif = false);
3
4 int main() {
5     motorHizAyarla(1);           // pwm = 128, frenAktif = false
6     motorHizAyarla(1, 200);      // pwm = 200, frenAktif = false
7     motorHizAyarla(1, 255, true); // pwm = 255, frenAktif = true
8     return 0;
9 }
```

- Sağdan Sola Kuralı: Varsayılan parametreler her zaman parametre listesinin en sağında yer almalıdır.
- Kural İhlali: `void f(int a = 10, int b);` **GEÇERSİZDİR** ve derleme hatası üretir!
- Tekrar Bildirimi Yasağı: Varsayılan değer prototipte verildiyse gövde tanımında tekrar yazılamaz.

Fonksiyon Aşırı Yükleme (Function Overloading)

Aynı amaca hizmet eden fonksiyonların aynı isim altında farklı parametrelerle tanımlanmasıdır:

```
1 // 1. İki tamsayıyı topla
2 int topla(int a, int b) {
3     return a + b;
4 }
5
6 // 2. İki kayan noktalı sayıyı topla
7 double topla(double a, double b) {
8     return a + b;
9 }
10
11 // 3. Üç tamsayıyı topla
12 int topla(int a, int b, int c) {
13     return a + b + c;
14 }
```

Derleyici Ayrımı: Derleyici çağrı yapılan noktadaki argümanların tipine ve sayısına bakarak hangi fonksiyonun çağrılacağını derleme zamanında (*static resolution*) çözer.

Fonksiyon İmzası (Function Signature)

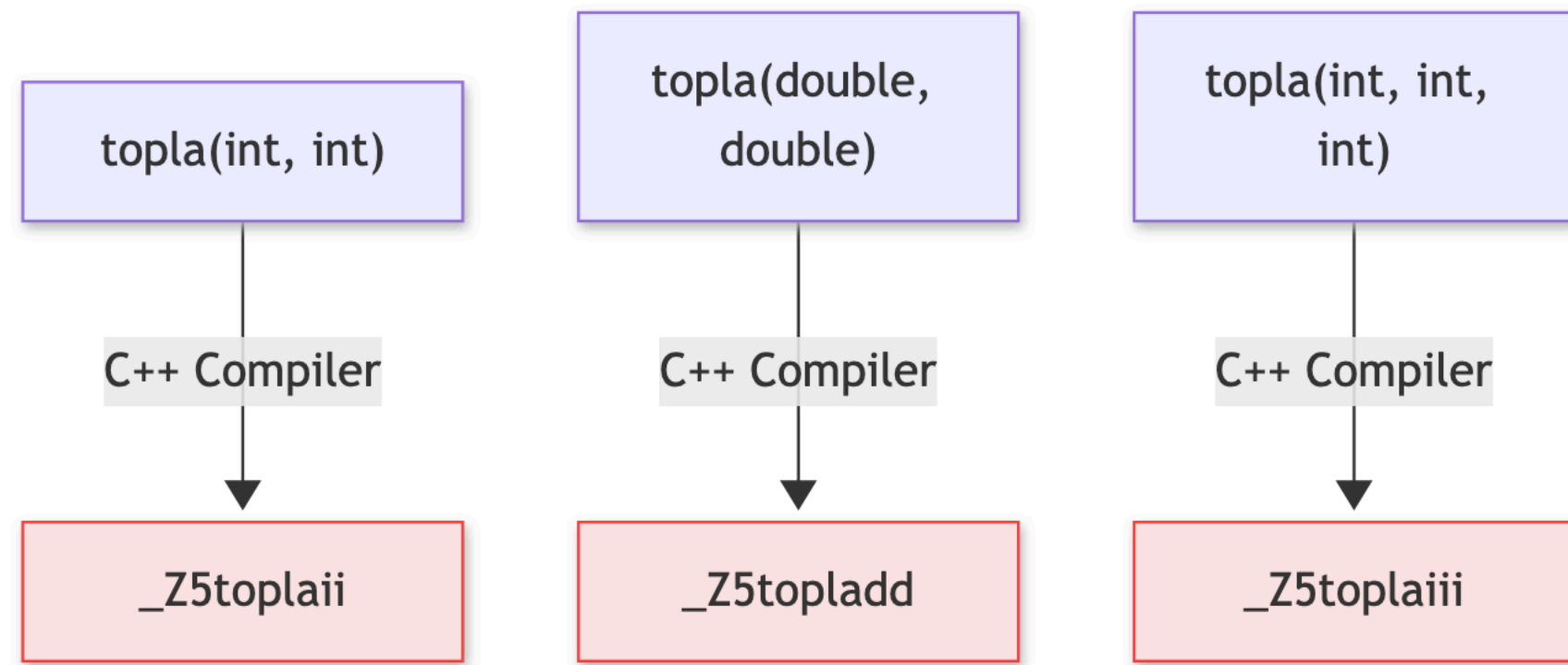
Bir fonksiyonun derleyici tarafından ayırt edilmesini sağlayan kimliktir:

- İmzaya Dahil Olanlar:
 - Fonksiyonun adı
 - Parametrelerin sayısı
 - Parametrelerin veri tipleri ve diziliş sırası
- İmzaya Dahil OLMAYANLAR:
 - Geri Dönüş Tipi: Yalnızca dönüş tipi farklı olan iki fonksiyon aşırı yüklenemez!
 - Parametre isimleri (`int x` ile `int y` aynı imzadır).

```
1 // GEÇERSİZ AŞIRI YÜKLEME (Derleme Hatası Verir):  
2 int veriOku(int pin);  
3 double veriOku(int pin); // HATA: Yalnızca dönüş tipi farklı!
```

Derleyici Arka Planı: İsim Bozma (Name Mangling)

C++ derleyicisi aynı isimli fonksiyonları bağlayıcı (linker) için nasıl ayrıştırır?



- **İsim Bozma (Name Mangling):** C++ derleyicisi fonksiyon adına parametre tiplerini ekleyerek küresel düzeyde benzersiz sembol isimleri türetir.
- **C Uyumluluğu (`extern "C"`):** C dilinde aşırı yükleme ve isim bozma yoktur. C kütüphanelerini C++'a bağlarken `extern "C"` bloğu kullanılır.

Aşırı Yüklemede Belirsizlik (Ambiguity) Riski

Derleyici birden fazla aşırı yüklenmiş fonksiyon arasında seçim yapamazsa hata verir:

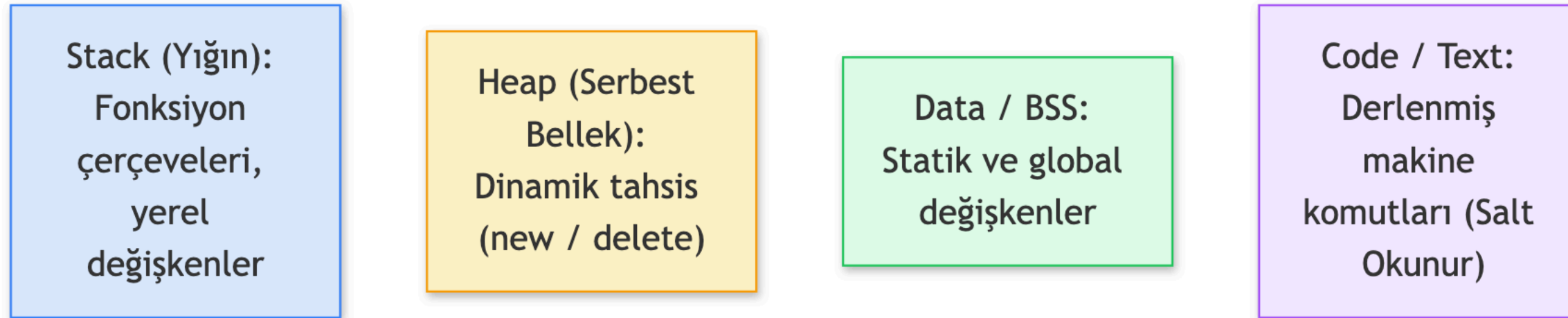
```
1 void yazdir(int x);
2 void yazdir(double x);
3
4 int main() {
5     yazdir(5);           // Net: yazdir(int) çağrılır
6     yazdir(3.14);       // Net: yazdir(double) çağrılır
7
8     // yazdir('A'); // Belirsizlik riski! char -> int mi, char -> double mı?
9     // yazdir(3.14f); // float -> double mı, float -> int mi?
10    return 0;
11 }
```

Çözüm: Tür dönüşümünü açıkça belirtmek için `static_cast` kullanılmalıdır:

```
yazdir(static_cast<double>(3.14f));
```

Bellek Anatomisi: Program Belleğinin 4 Bölgesi

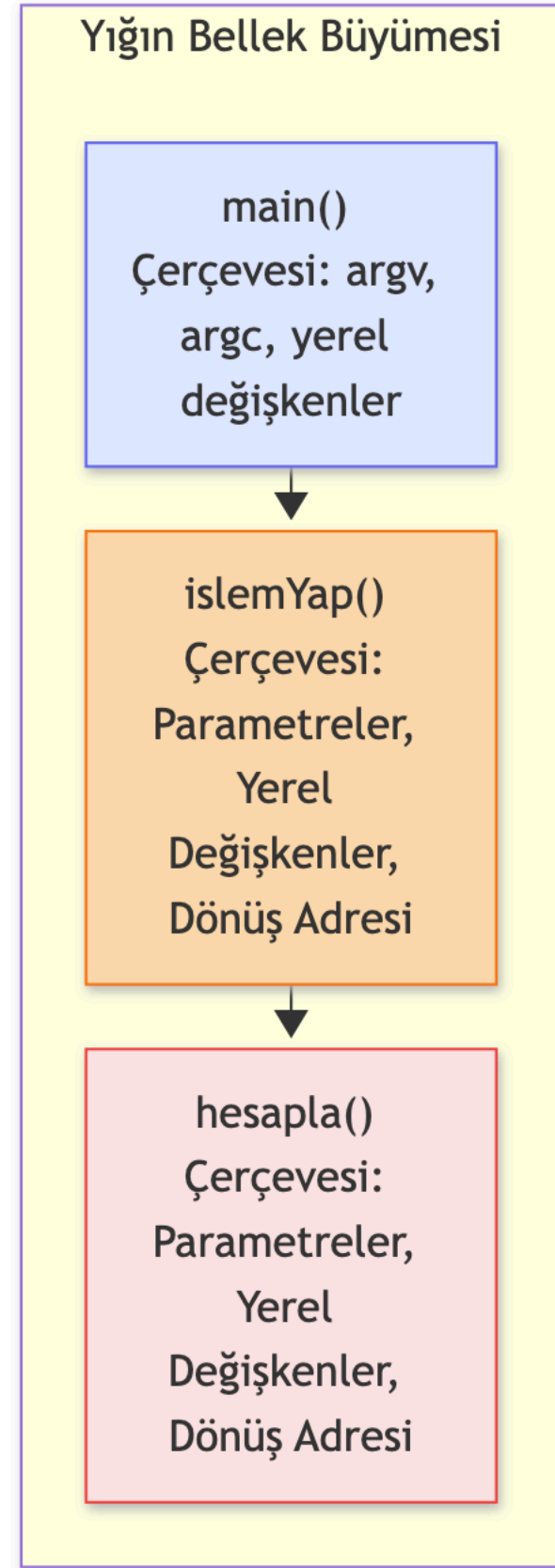
Çalışan bir C++ programı işletim sistemi tarafından tahsis edilen belleği 4 alana ayırır:



- **Stack Bölgesi:** LIFO (Last-In-First-Out) prensibiyle çalışan, son derece hızlı, otomatik yönetilen alandır.
- **Yığın Boyutu:** Genellikle kısıtlıdır (Linux/macOS'ta 8 MB, Windows'ta 1 MB varsayılan).

Çağrı Yığını Çerçevesi (Call Stack Frame)

Her fonksiyon çağrıldığında yığına bir Aktivasyon Kaydı (Stack Frame) itilir:



- Çerçeve İçeriği:
 - Fonksiyon parametreleri

Değişkenlerin Kapsamı (Scope) ve Ömrü (Lifetime)

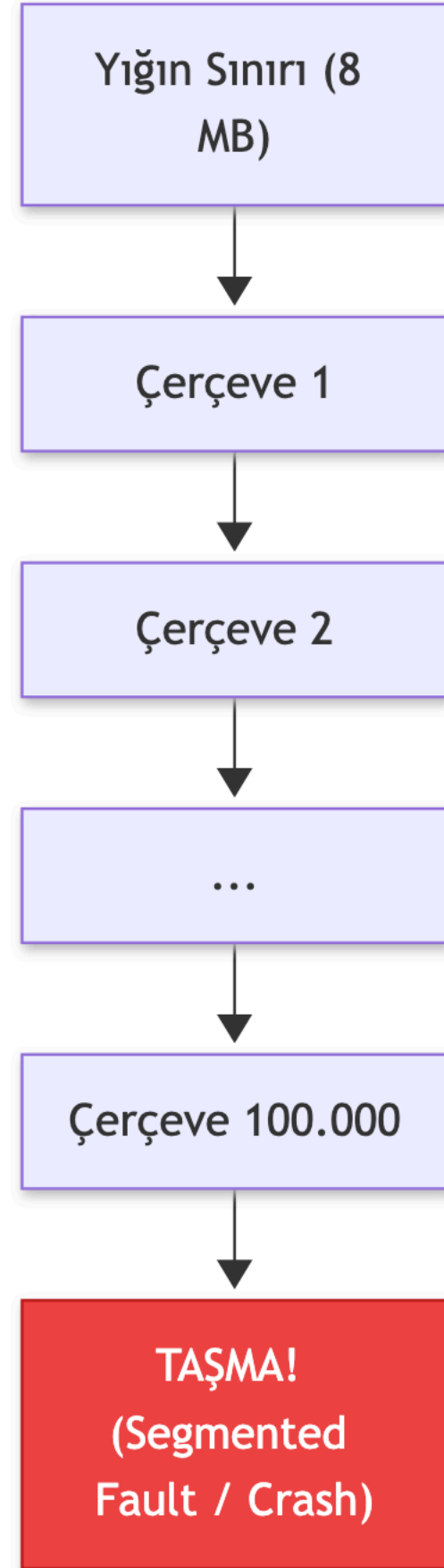
Yerel değişkenlerin ömrü tanımlandıkları blokla (`{ ... }`) sınırlıdır:

```
1 void test() {  
2     int yerel = 50; // Otomatik depolama süresi (Stack'te yaşar)  
3     std::cout << yerel << std::endl;  
4 } // 'yerel' değişkeni burada bellekten silinir  
5  
6 int* tehlikeliFonksiyon() {  
7     int x = 10;  
8     return &x; // ÖLÜMCÜL HATA! Fonksiyon bitince x yok olur!  
9             // Sarkan Gösterici (Dangling Pointer) üretir.  
10 }
```

Kritik Kural: Asla yerel bir değişkenin adresini veya referansını fonksiyondan geri döndürmeyiniz! Fonksiyon bittiğinde o bellek hücresi geçersiz hale gelir.

Yığın Taşması (Stack Overflow)

Yığın bellek alanı dolduğunda program işletim sistemi tarafından zorla kapatılır (*SIGSEGV*):



- Nedenler:

- Taban durumu eksik veya hatalı sonsuz özyineleme (*infinite recursion*).

`inline` Fonksiyonlar: Çağrı Yükünü Azaltma

Bir fonksiyon çağrıldığında CPU düzeyinde belirli bir zaman maliyeti (*overhead*) oluşur:

1. Parametreler yığına itilir.
2. Program sayacı fonksiyon adresine atlar (*jump / call*).
3. Fonksiyon bittiğinde yığın temizlenir ve geri dönülür (*ret*).

```
1 // Derleyiciye fonksiyon gövdesini çağrı noktasına gömme önerisi verir
2 inline double karesi(double x) {
3     return x * x;
4 }
5
6 int main() {
7     double sonuc = karesi(5.0);
8     // Derleme sonrasında makine kodu şuna dönüşür:
9     // double sonuc = 5.0 * 5.0; (Atlama maliyeti sıfırlandı!)
10    return 0;
11 }
```

inline Fonksiyonlar vs #define Makroları

C mirasından kalan önişlemci makroları yerine neden inline fonksiyonlar tercih edilmelidir?

Özellik	<code>#define KARE(x) ((x)*(x))</code>	<code>inline double kare(double x)</code>
Tip Güvenliği	YOK (Salt metin kopyalama)	TAM (Derleyici tip kontrolü yapar)
Yan Etki Güvenliği	TEHLİKELİ (<code>KARE(x++)</code> çift artırır!)	GÜVENLİ (Argüman bir kez değerlendirilir)
Hata Ayıklama	Sembol tablosunda görünmez	Adım adım hata ayıklanabilir (<i>debugger</i>)
Kapsam Kuralları	İsim alanı (<code>namespace</code>) tanımaz	Kapsam kurallarına tam uyar

Makro Tuzağı: `#define KARE(x) x * x` makrosuna `KARE(2 + 3)` gönderilirse `2 + 3 * 2 + 3 = 11` çıkar! `inline` fonksiyon ise `5 * 5 = 25` üretir.

Özyineleme (Recursion): Kendini Çağırان Fonksiyonlar

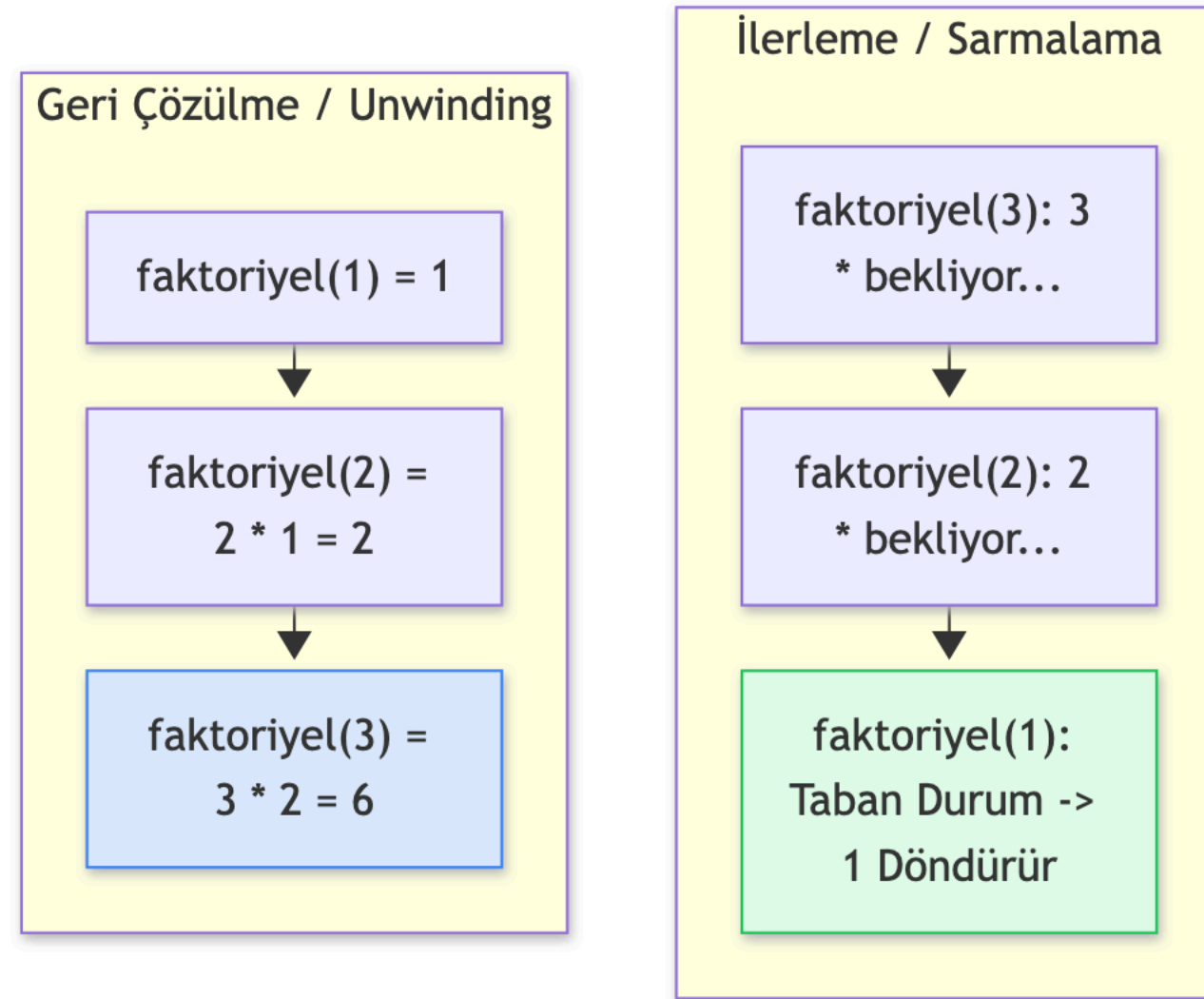
Bir problemin kendisinin daha küçük alt problemlerine bölünerek çözülmesidir:

- İki Temel Bileşen:
 - 1. Taban Durum (Base Case): Özyinelemenin durduğu, doğrudan cevabın döndüğü koşul. Taban durum yoksa program yığın taşmasıyla çöker.
 - 2. Özyinelemeli Adım (Recursive Step): Fonksiyonun problemi küçülterek kendini tekrar çağırdığı aşama.
- Matematiksel Karşılık: Tümevarım ve rekürans bağıntıları ($f(n) = n \cdot f(n - 1)$).

```
1 long long faktoriyel(int n) {  
2     if (n <= 1) return 1;           // Taban Durum (Base Case)  
3     return n * faktoriyel(n - 1);   // Özyinelemeli Adım  
4 }
```

Özyinelemede Çağrı Yığınının Görsel İlerlemesi

`faktoriyel(3)` çağrıldığında stack üzerinde gerçekleşen adımlar:



Bellek Maliyeti: Her özyinelemeli çağrı bellekte yeni bir yığın çerçevesi açar. $n=10.000$ için 10.000 adet çerçeve açılır; bu yüzden döngülü (iteratif) çözümler genellikle daha az bellek harcar.

Özyineleme Örneği: Fibonacci Serisi

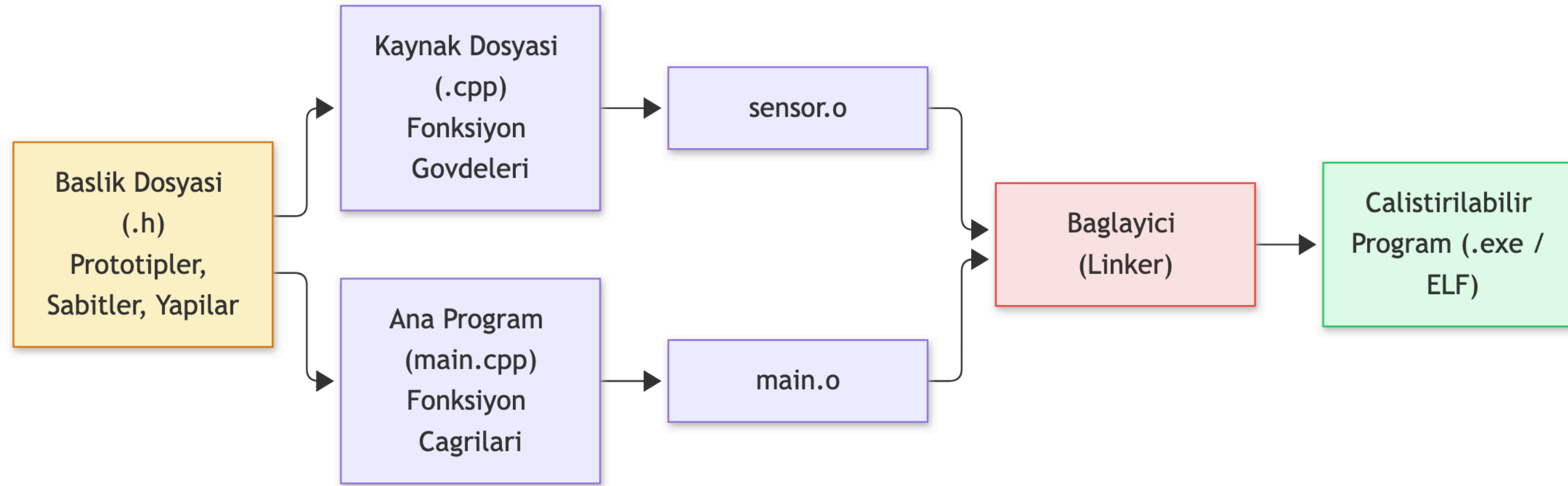
Fibonacci bağıntısı: $F(0) = 0, F(1) = 1, F(n) = F(n - 1) + F(n - 2)$

```
1  long long fibonacci(int n) {  
2      if (n <= 0) return 0; // Taban durum 1  
3      if (n == 1) return 1; // Taban durum 2  
4      return fibonacci(n - 1) + fibonacci(n - 2);  
5  }
```

- Verimsizlik Analizi: Bu saf özyinelemeli implementasyon $O(2^n)$ üssel zaman karmaşıklığına sahiptir!
- Aynı Değerlerin Tekrar Hesaplanması: `fib(5)` hesaplanırken `fib(3)` iki kez, `fib(2)` üç kez sıfırdan hesaplanır.
- Mühendislik Çıkarımı: Fibonacci gibi problemler için dinamik programlama veya $O(n)$ döngü tercih edilmelidir.

Modüler Programlama ve Çoklu Dosya Mimarisi

Büyük bir C++ projesi 3 ana bileşene bölünerek yönetilir:



Ayrık Derleme (Separate Compilation): Kaynak dosyalar birbirinden bağımsız derlenerek nesne dosyalarına (`.o` / `.obj`) çevrilir.

Değişen tek bir `.cpp` tüm projenin baştan derlenmesini gerektirmez.

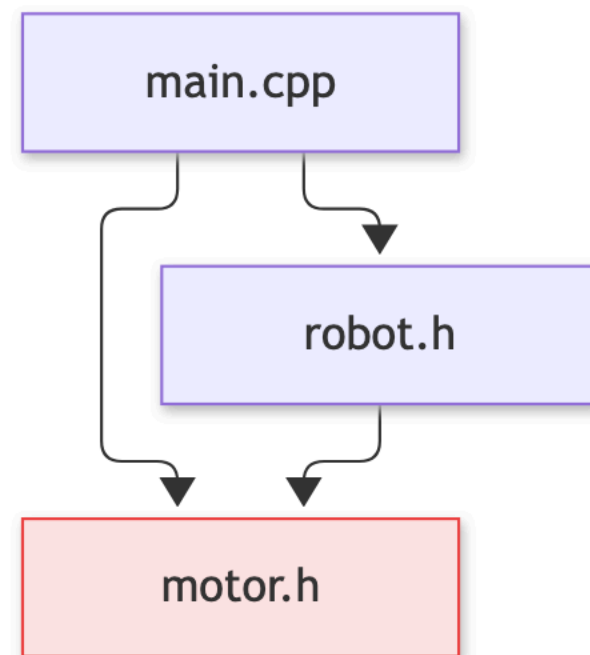
Başlık Dosyası (`.h`) Mimarisi

Başlık dosyaları projenin diğer parçalarıyla paylaşılan ortak arayüz sözleşmesidir:

- Başlık Dosyasına (`.h`) NELER KONUR?
 - Fonksiyon prototipleri (deklarasyonlar)
 - Sabit tanımları (`const` , `constexpr`)
 - Veri yapıları ve sınıf tanımları (`struct` , `class` , `enum class`)
 - `inline` fonksiyon tanımları ve şablonlar (*templates*)
- Başlık Dosyasına NELER KONMAZ?
 - Normal fonksiyon gövdeleri (bağlayıcıda çoklu tanım - *multiple definition* hatası verir!)
 - Değişken tanımları (örneğin `int sayac = 0;` konulursa her dahil edildiğinde kopyalanır)
 - `using namespace std;` direktifi (diğer tüm dosyaların isim alanını kirletir!)

Başlık Muhafızları (Include Guards)

Bir başlık dosyası projede farklı kaynak dosyalarınca defalarca `#include` edilebilir:



Eğer önlem alınmazsa `motor.h` iki kez dahil edilir ve "yeniden tanımlama" (*redefinition*) hatası oluşur!

```
1 // motor.h dosyasının korunması
2 #ifndef MOTOR_H // MOTOR_H daha önce tanımlanmadıysa içeri gir
3 #define MOTOR_H // MOTOR_H sembolünü tanımla
4
5 void motorHizAyarla(int pwm);
6
7 #endif // MOTOR_H
```

Modern Alternatif: `#pragma once`

Modern derleyicilerde geleneksel `#ifndef` makroları yerine tek satırlık direktif kullanılır:

```
1 // motor.h (Modern C++ Yaklaşımı)
2 #pragma once
3
4 void motorHizAyarla(int pwm);
```

- Avantajları:
 - Tek satırdır, yazımı ve okunması son derece kolaydır.
 - Makro ismi çakışması (*macro name collision*) riskini sıfırlar.
 - Derleyici dosyayı bir kez okuduktan sonra diski tekrar taramaz (derleme hızını artırır).
- Standart Durumu: C++ standardının resmi bir parçası değildir; ancak GCC, Clang, MSVC gibi tüm modern derleyiciler tarafından eksiksiz desteklenir.

Modüler Kod Tasarımı: robot_kinematik.h

Mekatronik robot kolu açısı dönüşümleri için başlık dosyası tasarımı:

```
1  #pragma once
2
3  // Matematiksel Sabitler
4  constexpr double PI = 3.14159265358979323846;
5
6  // Veri Yapısı
7  struct EklemAcilari {
8      double eklem1;
9      double eklem2;
10     double eklem3;
11 };
12
13 // Fonksiyon Prototipleri
14 double dereceToRadyan(double derece);
15 double radyanToDerece(double radyan);
16 bool kinematikLimitKontrol(const EklemAcilari& acilar, double minAci, double maxAci);
```

Modüler Kod Tasarımı: `robot_kinematik.cpp`

Prototiplerin gövde tanımları kaynak dosyada gerçekleştirilir:

```
1  #include "robot_kinematik.h"
2
3  double dereceToRadyan(double derece) {
4      return derece * (PI / 180.0);
5  }
6
7  double radyanToDerece(double radyan) {
8      return radyan * (180.0 / PI);
9  }
10
11 bool kinematikLimitKontrol(const EklemAcilari& acilar, double minAci, double maxAci) {
12     if (acilar.eklem1 < minAci || acilar.eklem1 > maxAci) return false;
13     if (acilar.eklem2 < minAci || acilar.eklem2 > maxAci) return false;
14     if (acilar.eklem3 < minAci || acilar.eklem3 > maxAci) return false;
15     return true;
16 }
```

Modüler Kod Tasarımı: `main.cpp`

Ana uygulama başlık dosyasını dahil ederek fonksiyonları güvenle kullanır:

```
1  #include <iostream>
2  #include "robot_kinematik.h"
3
4  int main() {
5      EklemAcilari hedef = {45.0, 90.0, -30.0};
6
7      if (kinematikLimitKontrol(hedef, -90.0, 90.0)) {
8          std::cout << "Eklem 1 (Radyan): " << dereceToRadyan(hedef.eklem1) << std::endl;
9          std::cout << "Eklem 2 (Radyan): " << dereceToRadyan(hedef.eklem2) << std::endl;
10     } else {
11         std::cout << "HATA: Eklem acilari limit disinda!" << std::endl;
12     }
13     return 0;
14 }
```

Çok Dosyalı Projelerin Terminalden Derlenmesi

Çoklu kaynak dosyaları derleyiciden tek komutla veya ayrı ayrı derlenebilir:

- 1. Tek Adımda Doğrudan Derleme:

```
1 g++ -std=c++14 -Wall main.cpp robot_kinematik.cpp -o robot_kontrol
```

- 2. Profesyonel Ayrık Derleme (Separate Compilation):

```
1 # Adım 1: Kaynak dosyaları nesne dosyalarına derle (-c)
2 g++ -std=c++14 -c robot_kinematik.cpp -o robot_kinematik.o
3 g++ -std=c++14 -c main.cpp -o main.o
4 # Adım 2: Nesne dosyalarını bağlayıcı ile birleştir
5 g++ main.o robot_kinematik.o -o robot_kontrol
```

Önemli Kural: Başlık dosyaları (`.h`) derleyici komut satırına parametre olarak yazılmaz! Yalnızca `.cpp` dosyaları derlenir; `.h` dosyaları `#include` ile dahil edilir.

Laboratuvar Aıştırması 1: Robot 3 Eksen Konumlandırma

Problem: 3 eksenli (X, Y, Z) bir Kartezyen robotun konum koordinatlarını referans parametrelerle güncelleyen ve limit aşımını denetleyen bir fonksiyon kütüphanesi kodlayınız.

- Fonksiyon İmzası:

```
1  bool konumaGit(int& guncelX, int& guncelY, int& guncelZ,  
2          int hedefX, int hedefY, int hedefZ,  
3          int limit);
```

- Gereksinimler:

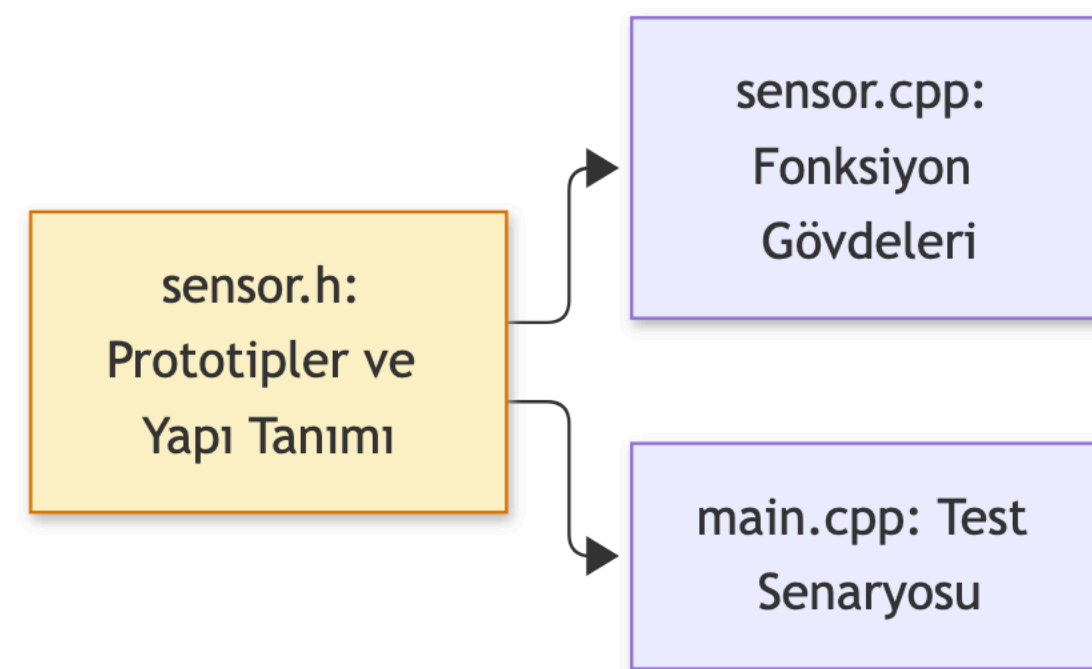
- Hedef koordinatlar `[-limit, +limit]` aralığında değilse fonksiyon `false` döndürmeli, koordinatlar değişmemelidir.
- Değerler güvenliyse referans parametreler üzerinden güncellenmeli ve `true` döndürülmelidir.

Alıştırma 1: Çözüm Kodu

```
1  #include <iostream>
2
3  bool konumaGit(int& curX, int& curY, int& curZ,
4               int targetX, int targetY, int targetZ, int limit) {
5      if (std::abs(targetX) > limit || std::abs(targetY) > limit || std::abs(targetZ) > limit)
6          return false; // Limit asildi
7      }
8      curX = targetX;
9      curY = targetY;
10     curZ = targetZ;
11     return true;
12 }
13
14 int main() {
15     int x = 0, y = 0, z = 0;
16     if (konumaGit(x, y, z, 150, -80, 40, 200)) {
17         std::cout << "Yeni Konum: (" << x << ", " << y << ", " << z << ")\n";
18     }
19     return 0;
}
```

Laboratuvar A1ştırması 2: Akıllı Sensör Telemetry Modülü

Problem: Modüler yapıda (`sensor.h` , `sensor.cpp` , `main.cpp`) çalışan bir sıcaklık ve nem alarm filtreleme sistemi geliştiriniz.



- Gereksinimler:
 - `SensorVerisi` yapısı: `double sıcaklik` , `double nem` .
 - `veriGecerliMi(const SensorVerisi& v)` : Sıcaklık $[-40, 85]$, nem $[0, 100]$ aralığında mı kontrol eder (`const&`).
 - `alarmKontrol(const SensorVerisi& v, double sıcaklikEsik, double nemEsik)` : Eşik aşımında alarm bayrağı döner.

Alıştırma 2: Başlık Dosyası (`sensor.h`)

```
1  #pragma once
2
3  struct SensorVerisi {
4      double sicaklik;
5      double nem;
6  };
7
8  // Fonksiyon Prototipleri
9  bool veriGecerliMi(const SensorVerisi& v);
10 bool alarmKontrol(const SensorVerisi& v, double sicaklikEsik, double nemEsik);
11 void telemetriYazdir(const SensorVerisi& v);
```

Standart: Fonksiyon parametrelerinde büyük veri yapılarının gereksiz kopyalanmasını önlemek için `const SensorVerisi&` kullanılmıştır.

Alıştırma 2: Fonksiyon Tanımları (sensor.cpp)

```
1  #include <iostream>
2  #include "sensor.h"
3
4  bool veriGecerliMi(const SensorVerisi& v) {
5      if (v.sicaklik < -40.0 || v.sicaklik > 85.0) return false;
6      if (v.nem < 0.0 || v.nem > 100.0) return false;
7      return true;
8  }
9
10 bool alarmKontrol(const SensorVerisi& v, double sicaklikEsik, double nemEsik) {
11     return (v.sicaklik > sicaklikEsik || v.nem > nemEsik);
12 }
13
14 void telemetriYazdir(const SensorVerisi& v) {
15     std::cout << "[TELEMETRI] Sicaklik: " << v.sicaklik
16               << " C | Nem: %" << v.nem << "\n";
17 }
```

Alıştırma 2: Ana Program (`main.cpp`)

```
1  #include <iostream>
2  #include "sensor.h"
3
4  int main() {
5      SensorVerisi paket1 = {34.5, 62.0};
6      SensorVerisi paket2 = {95.0, 20.0}; // Hatalı sıcaklık
7
8      if (veriGecerliMi(paket1)) {
9          telemetriYazdir(paket1);
10         if (alarmKontrol(paket1, 30.0, 50.0)) {
11             std::cout << "UYARI: Paket 1 kritik esik degerleri asti!\n";
12         }
13     }
14
15     if (!veriGecerliMi(paket2)) {
16         std::cout << "HATA: Paket 2 sensor limitleri disinda (Gecersiz Veri)!\n";
17     }
18     return 0;
19 }
```

Laboratuvar Alıştırması 3: Rekürsif Güç ve Tork Filtresi

Problem: Mekatronik bir sistemde bir sayının üssünü ($taban^{us}$) özyinelemeli olarak hesaplayan fonksiyonu ve bunun negatif üsler için aşırı yüklenmiş (`overloaded`) halini tasarlayınız.

```
1  #include <iostream>
2
3  // Pozitif tamsayı üsler için özyinelemeli fonksiyon
4  double usAl(double taban, int us) {
5      if (us == 0) return 1.0;           // Taban durum
6      if (us < 0) return 1.0 / usAl(taban, -us); // Negatif üs dönüşümü
7      return taban * usAl(taban, us - 1); // Rekürsif adım
8  }
9
10 int main() {
11     std::cout << "2^8   = " << usAl(2.0, 8) << "\n"; // 256
12     std::cout << "2^-3 = " << usAl(2.0, -3) << "\n"; // 0.125
13     return 0;
14 }
```

Hafta 3: Özet ve Mühendislik Kazanımları

Bu derste ele alınan kritik ilkeler:

- Prototip ve Modülerlik: Kodlarımızı `.h` (arayüz/sözleşme) ve `.cpp` (implementasyon) olarak ayırarak derleme süresini ve modülerliği optimize ettik.
- Bellek Güvenliği ve Performans:
 - Küçük türler için Değerle Aktarım (`T`)
 - Büyük veri yapıları ve nesneler için `const` Referansla Aktarım (`const T&`)
 - Çıktı parametreleri için Referansla Aktarım (`T&`)
- Yığın (Stack) Bilinci: Fonksiyon çağrılarının aktivasyon kayıtları oluşturduğunu, yığın sınırlarının aşılabileceğini (*stack overflow*) ve yerel referansların asla geri döndürülmeyeceğini öğrendik.
- Gelecek Hafta: Diziler, C-stringler, `std::string` sınıfı ve modern `std::vector` konteynerine giriş yapacağız.

Kaynaklar ve İletişim

- Ders Deposu: github.com/marmara-mekatronik/MRM3049
- Kaynak Kitap: Deitel & Deitel, *C++ How to Program* (Bölüm 6: Functions and an Introduction to Recursion)
- İletişim: Dr. Hüseyin Yüce (Marmara Üniversitesi Teknoloji Fakültesi / Mekatronik Mühendisliği)

Nesne Yönelimli Programlama • Hafta 3: Fonksiyonlar, Bellek Mimarisi ve Modülerlik